

Oficina 1 — Painel de atuadores

Ligar quatro coisas e descobrir o que ainda não dá para fazer

Microcontroladores — DENE/UFMT

SEM NOTA

Esta oficina não vale nota. Ela existe para que vocês toquem em todos os atuadores do projeto antes que qualquer um deles precise ser controlado, e para que cheguem ao Roteiro 3 com uma pergunta já formada. Traga o registro preenchido — ele será discutido em sala, não corrigido.

NOTA

Continuidade com a aula de hoje. A aula terminou com uma ideia: a arquitetura decide coisas por vocês, e o programa tem de caber nelas. Esta oficina é a primeira vez que isso deixa de ser afirmação e passa a ser observável — em quatro atuadores, com cronômetro.

Os conceitos usados aqui foram vistos há trinta minutos e não serão reexplicados: $T_{cy} = 4/f_{osc}$ e o núcleo a 16 MHz (§2 da aula), os registradores `TRIS` / `LAT` / `PORT` e a ordem `LAT` antes de `TRIS` (§4.1), e o fato de o bootloader ser o dono das palavras de configuração (§2.2).

OBJETIVOS

- Acionar, um a um, os quatro atuadores da bancada usando apenas saída digital.
- Escrever previsões antes de cada experimento e confrontá-las com o observado.
- Verificar experimentalmente por que `LAT` é escrito antes de `TRIS`.
- Constatar o limite do acionamento liga/desliga e enunciar o problema que o Roteiro 3 resolve.

1 Antes de ligar a bancada

PERIGO

Os pontos de teste **LAMP**, **HEATER** e **COOLER** estão no trilho de **12 V**, não no de 5 V do microcontrolador.

- **Não** prenda a garra de terra do osciloscópio nesses pontos. Já houve curto-circuito nesta disciplina por esse motivo.
- **Não** remova o pino de terra de proteção do osciloscópio como contorno. Isso não resolve o problema de referência e cria risco de choque.
- Nesta oficina não há medição com osciloscópio. Observação, ouvido e cronômetro bastam.

ATENÇÃO

A lâmpada é o aquecedor do projeto e aquece de verdade. Mantenha-a ligada por **no máximo 30 segundos** por vez nesta oficina, e não a toque depois de desligar. O ciclo térmico é lento: não vale a pena esperar esfriar entre experimentos.

BANCADA

Por bancada: kit XM118 com fonte, cabo USB, computador com MPLAB X e XC8 instalados, aplicativo *Bootloader PIC18/XM118 v2.8*, cronômetro (o do celular serve).

Recuperação do R0: o código de referência do Roteiro 0 está publicado no repositório da disciplina. Se a bancada não estiver piscando o LED ao fim dos primeiros dez minutos, chame o professor em vez de depurar às cegas.

1.1 Gravação: o que muda em relação ao R0

Nada. O procedimento é o mesmo, e vale repetir porque é onde se perde tempo:

1. No MPLAB X, o projeto compila em modo *No Tool*. O MPLAB X **não** grava o dispositivo nesta disciplina.
2. Pressione **SW9** na placa e, dentro de uma janela de **4 a 5 segundos**, abra o aplicativo do bootloader e envie o `.hex`.
3. Se a janela passar, pressione SW9 de novo. Não há prejuízo em repetir.

ATENÇÃO

O manual da Exsto indica **SW1** como botão de entrada em modo de gravação. Está errado: é **SW9**. Sempre que o manual e a serigrafia da placa discordarem, a serigrafia tem precedência.

2 Previsão

Esta tabela foi preenchida há trinta minutos, no fim da aula teórica. Tenha-a à mão: cada experimento adiante remete a uma das linhas, e o que interessa é o confronto entre o previsto e o observado.

NOTA

Previsão errada não é problema. Previsão ausente é: sem ela, o experimento vira demonstração, e demonstração não ensina nada que a foto de um LED aceso já não ensinasse. O que se avalia ao longo do semestre é a qualidade do raciocínio, nunca o acerto.

Quem chegou sem a folha preenche agora, antes de compilar qualquer coisa — e sem consultar o colega da bancada ao lado.

Pergunta	Previsão e justificativa	Rubrica
P1. O que acontece com o cooler no instante do <i>reset</i> , antes de a primeira linha do seu programa executar?		
P2. Se <code>TRIS</code> for configurado antes de <code>LAT</code> , o comportamento muda? Como?		
P3. Piscando o cooler a 10 Hz, o que você espera ver e ouvir?		
P4. E a 1 kHz?		
P5. Piscar mais rápido faz o cooler girar mais devagar? Justifique.		

3 O programa base

DIVERGÊNCIA

O bloco de **#define** abaixo é provisório. Os pinos que acionam LEDs, buzzer, lâmpada e cooler no XM118 devem ser lidos do bloco congelado do **Roteiro 0** e conferidos contra a **serigrafia da placa**. Substitua o bloco inteiro antes de usar este roteiro em bancada — nenhuma outra parte do código depende de pinos literais.

Segue pendente também a ambiguidade de numeração das chaves DIP (**CH4-7** ou **CH4-8** para RC7/RX), a ser resolvida por inspeção física.

```
/* -----  
Oficina 1 – painel de atuadores  
Acionamento digital simples. Nenhum controle ainda.  
----- */  
  
#define _XTAL_FREQ 16000000UL /* nucleo a 16 MHz – ver Aula 1 */  
  
#include <xc.h>  
#include <stdint.h>  
  
/* --- mapeamento da bancada: SUBSTITUIR pelo bloco do R0 ----- */  
#define LEDS_LAT LATD  
#define LEDS_TRIS TRISD  
  
#define BUZZER_LAT LATCbits.LATC1  
#define BUZZER_TRIS TRISCbits.TRISC1  
  
#define LAMPADA_LAT LATCbits.LATC0  
#define LAMPADA_TRIS TRISCbits.TRISC0  
  
#define COOLER_LAT LATCbits.LATC2  
#define COOLER_TRIS TRISCbits.TRISC2  
/* ----- */  
  
static void atuadores_init(void)  
{  
    /* 1) estado seguro PRIMEIRO: tudo desligado */  
    LEDS_LAT = 0x00;  
    BUZZER_LAT = 0;  
    LAMPADA_LAT = 0;  
    COOLER_LAT = 0;  
  
    /* 2) so entao habilitar as saidas */  
    LEDS_TRIS = 0x00;  
    BUZZER_TRIS = 0;  
    LAMPADA_TRIS = 0;  
    COOLER_TRIS = 0;  
}  
  
void main(void)  
{  
    atuadores_init();  
  
    while (1) {  
        /* os experimentos entram aqui, um de cada vez */  
    }  
}
```

NOTA

Não há bloco `#pragma config` neste programa, e isso é intencional. Na XM118 quem fixa as palavras de configuração é o **bootloader**, que já está gravado no dispositivo. Diretivas `#pragma config` escritas na aplicação são ignoradas em silêncio — compilam, não reclamam e não têm efeito. A Aula 1 trata do assunto; por ora, basta saber que a frequência do núcleo está fixada em 16 MHz e que `_XTAL_FREQ` precisa refletir isso, ou todos os atrasos sairão errados por um fator três.

4 Acionamento, um atuador de cada vez

Faça na ordem. Cada item acrescenta uma linha ou duas ao laço principal; não apague o anterior sem antes anotar o resultado.

4.1 Os LEDs

TAREFA

Acenda a barra inteira de LEDs. Depois faça-a piscar a 1 Hz.

```
while (1) {  
    LEDS_LAT = 0xFF;  
    __delay_ms(500);  
    LEDS_LAT = 0x00;  
    __delay_ms(500);  
}
```

Cronometre **20 piscadas completas** e divida por 20. Se o período medido não bater com 1 s dentro de uns poucos por cento, o valor de `_XTAL_FREQ` está errado — e essa é a primeira evidência de clock que vocês terão.

Vocês já fizeram isso por conta própria na sessão passada. A diferença aqui é a medição: sem cronômetro, “piscou” e “piscou no ritmo certo” são a mesma observação.

4.2 O buzzer

TAREFA

Acione o buzzer com pulsos curtos — nunca contínuo.

```
for (uint8_t i = 0; i < 5; i++) {  
    BUZZER_LAT = 1;  
    __delay_ms(60);  
    BUZZER_LAT = 0;  
    __delay_ms(200);  
}  
__delay_ms(3000);
```

ATENÇÃO

Doze bancadas com buzzer contínuo tornam a sala inutilizável. Pulsos curtos, e desligue assim que registrar o resultado.

4.3 A lâmpada

TAREFA

Ligue a lâmpada por 10 segundos, desligue e anote: quanto tempo o filamento leva para apagar completamente depois que `LAMPADA_LAT` vai a zero?

```
LAMPADA_LAT = 1;  
__delay_ms(10000);  
LAMPADA_LAT = 0;
```

EXPERIMENTO

A lâmpada não apaga no instante em que o pino vai a zero, e o sensor de temperatura vai demorar bem mais que isso para acusar a mudança. Esse atraso entre **comandar** e **observar o efeito** é a razão de existir do controle com histerese no Roteiro 9 — e é a primeira vez que ele aparece de forma visível. Anote a ordem de grandeza que você observou.

4.4 O cooler

TAREFA

Ligue o cooler, deixe estabilizar, desligue. Anote quanto tempo ele leva para parar por inércia.

```
COOLER_LAT = 1;
__delay_ms(5000);
COOLER_LAT = 0;
```

NOTA

Com o pino em nível alto o cooler gira na velocidade máxima. Com o pino em zero, ele para. Não há terceira opção neste programa — e é exatamente esse o ponto da seção 5.

5 A ordem que importa: LAT antes de TRIS

Até aqui, a função `atuadores_init` escreveu `LAT` antes de `TRIS` sem justificativa. Agora inverta e observe.

EXPERIMENTO

Troque a ordem dos dois blocos de `atuadores_init`:

```
static void atuadores_init(void)
{
    COOLER_TRIS = 0;      /* saída habilitada ANTES ... */
    LAMPADA_TRIS = 0;
    BUZZER_TRIS = 0;
    LEDS_TRIS    = 0x00;

    COOLER_LAT   = 0;      /* ... do estado ser definido */
    LAMPADA_LAT  = 0;
    BUZZER_LAT   = 0;
    LEDS_LAT     = 0x00;
}
```

Grave e pressione o botão de *reset* várias vezes seguidas, prestando atenção no cooler e no buzzer. Depois volte à ordem original e repita. Registre a diferença.

CONCEITO

No instante do *reset*, todo pino nasce como **entrada** (`TRIS = 1`) e o conteúdo de `LAT` é **indefinido**. Enquanto o pino é entrada, o valor de `LAT` não chega ao mundo: o pino está em alta impedância e o cooler permanece desligado.

Escrever `TRIS = 0` habilita a saída — e nesse exato instante o pino passa a refletir o que quer que esteja em `LAT`. Se `LAT` ainda não foi escrito, o atuador é comandado por lixo, por alguns microssegundos ou milissegundos, até que a linha seguinte corrija.

Daí a regra que vale para todo atuador do semestre: **defina o estado seguro em LAT , depois habilite a saída em TRIS .** Com um LED, o transitório é invisível. Com um motor e uma resistência de aquecimento, não é.

ATENÇÃO

Essa é a razão de o Roteiro 0 insistir na ordem antes de haver qualquer atuador de potência ligado. Um hábito adquirido com o LED é o que evita o tranco no cooler quando o hábito passa a importar.

6 O fracasso interessante: meia velocidade

O cooler tem duas velocidades no programa atual: tudo e nada. A pergunta óbvia é como obter algo no meio. A tentativa igualmente óbvia é piscar rápido.

EXPERIMENTO

Rodada A — 10 Hz. Cinquenta milissegundos ligado, cinquenta desligado.

```
while (1) {  
    COOLER_LAT = 1;  
    __delay_ms(50);  
    COOLER_LAT = 0;  
    __delay_ms(50);  
}
```

Observe e escute. O cooler gira de forma constante ou aos solavancos?

Rodada B — 1 kHz. Quinhentos microssegundos de cada lado.

```
while (1) {  
    COOLER_LAT = 1;  
    __delay_us(500);  
    COOLER_LAT = 0;  
    __delay_us(500);  
}
```

Agora sim há uma velocidade intermediária estável. Anote-a como referência.

Rodada C — mude a proporção. Mantenha o período de 1 ms, mas use 200 μ s ligado e 800 μ s desligado. Depois 800 e 200. Registre as três velocidades observadas.

A rodada C funciona: dá para escolher a velocidade. E é justamente por funcionar que o problema fica visível.

TAREFA

Responda por escrito, antes de sair:

1. Para mudar a velocidade do cooler neste programa, o que precisa ser alterado? Basta mudar um número em tempo de execução, ou é preciso reescrever e regravar?
2. Enquanto o laço da rodada C executa, o que mais o microcontrolador está fazendo?
3. Some os atrasos: quanto tempo de CPU sobra, por segundo, para ler um sensor de temperatura e decidir alguma coisa?

CONCEITO

As respostas convergem para o mesmo lugar. O laço de atraso ocupa **cem por cento** do tempo do processador: não sobra ciclo nenhum para ler o sensor, atualizar o display ou responder a um botão. E a

proporção entre ligado e desligado está congelada dentro de chamadas de `__delay_us`, isto é, no código compilado — não é uma variável que um controlador possa ajustar.

Um termostato precisa exatamente das duas coisas que faltam: variar a proporção continuamente, em função da temperatura lida, e fazer isso **sem** parar de executar o resto do programa.

É esse o problema que o Roteiro 3 resolve, transferindo a geração do sinal para um periférico dedicado que trabalha sozinho enquanto a CPU cuida de outra coisa. O nome disso é **modulação por largura de pulso**, e o módulo que a implementa no PIC18F4550 chama-se CCP1.

NOTA

Vale registrar o que acabou de acontecer. Vocês não receberam o PWM como conceito a memorizar: chegaram a ele por eliminação, tendo esgotado o que a saída digital pura permite. Quando o Roteiro 3 introduzir os registradores do CCP1, eles não serão uma novidade arbitrária — serão a resposta a uma pergunta que já era de vocês.

7 Registro

Traga na próxima sessão, em uma folha ou no caderno de bancada:

Item	O que registrar
Previsões	A tabela da seção 2, preenchida e rubricada antes dos experimentos
Período medido	Tempo de 20 piscadas dos LEDs, dividido por 20, e o desvio em relação a 1 s
Lâmpada	Tempo aproximado até o filamento apagar completamente
Cooler	Tempo de parada por inércia após desligar
Ordem de inicialização	O que mudou entre as duas versões de <code>atuadores_init</code> , no cooler e no buzzer
Rodadas A, B, C	Comportamento em cada uma; as três velocidades da rodada C
Respostas	As três perguntas da seção 5
Divergências	Qualquer discordância entre este roteiro, o manual da Exsto e a placa

SEM NOTA

Novamente: nada disso vale nota. O registro serve à discussão de abertura da próxima sessão e ao seu próprio proveito quando o Roteiro 3 chegar.